Labyrinth Breach

Reinforcement Learning-based Multi-Agent Pursuit-Evasion in Dynamic Maze Environments

Muhammad Sikander Raheem Usman Irshad Bhatti Ahmed Jawed

MS AI, LUMS | AI641 – AI for Robotics

May 2025

Unity ML-Agents | PPO | MARL

Project Overview

What is Labyrinth Breach?

- **Environment:** Unity ML-Agents maze simulator
- **Setup:** 3 Sentinels vs 2 Runners (3v2 asymmetric)
- **Goal (Sentinels):** Capture both Runners before timeout
- **Goal (Runners):** Reach an exit zone or survive 120s
- **Mazes:** Procedurally generated with dynamic wall shifts

Key Technical Challenges

- **Partial Observability** – Raycasts + last-known-position memory; no global state
- **Multi-Agent Coordination** – 3 agents must cooperate to trap 2 evaders
- **Dynamic Environment** – Walls shift mid-episode, forcing real-time adaptation
- **Asymmetric Teams** – Different team sizes, roles, reward structures
- **Generalization** – Must work on unseen maze layouts

Literature Review – Foundations & Related Work

Pursuit-Evasion

Route blocking, pincer, spatial entrapment, escape timing

Cooperative MARL

Non-stationarity, CTDE, parameter sharing, PPO baselines

Credit Assignment

Shared reward too coarse, counterfactual baselines, reward shaping

Partial Observability

LOS constraints, memory, uncertainty changes optimal strategies

Theme 1 – Pursuit-Evasion as a Multi-Agent Problem

Core Concept

Classical robotics and MARL problem: one side captures, the other evades. RL is a natural fit because desired behavior is hard to hand-code under adversarial interaction, obstacles, and partial information.

Richer Behaviors Required

Sentinel:

- Trapping and pincer
- Corridor denial
- Exit blocking
- Route cutting

Runner:

- Distance management
- Path selection
- Survival timing
- Exploit topology

Key References

Verma et al. (2019) – Multi-agent pursuer-evader learning; emergent route-blocking from simple rewards.

Yan et al. (2019) – Pursuit-evasion with uncertain pursuer speeds; coordination under real-world constraints.

Liu et al. (2024) – Multi-UAV pursuit-evasion via PSO-M3DDPG; large-scale aerial coordination.

Haeri et al. (2022) – Emergent behaviors in multi-agent target acquisition; rich behavior from simple reward signals.

Theme 2 – Cooperative & Mixed Multi-Agent RL

Core Challenge: Non-Stationarity – All agents update simultaneously; each agent's environment is non-stationary, making standard RL convergence guarantees inapplicable.

MADDPG (Lowe et al., 2017)

Centralized Training, Decentralized Execution (CTDE): actor uses local obs, critic accesses global state.

Our use: CTDE principle; we use PPO's parameter sharing as a simpler alternative.

MAPPO (Yu et al., 2022)

Well-tuned PPO surprisingly outperforms complex algorithms in cooperative MARL. Lower complexity, better reproducibility.

Our use: Primary training method; Unity ML-Agents provides native PPO support.

Parameter Sharing

Agents within a team share a single policy network. Reduces sample complexity and enforces role symmetry.

Our use: 3 Sentinels share one policy; 2 Runners share another – distinct from each other.

Theme 3 – Credit Assignment in Cooperative MARL

The Credit Assignment Problem

When multiple agents cooperate under team-level objectives, one may contribute by blocking a route while another captures. Plain shared reward is too coarse – agents cannot distinguish meaningful contributions from free-riding.

Our Approach

Tactical reward shaping with per-agent bonuses for coordination events: pincer formations, corridor control, exit denial, and dead-end forcing.

Landmark Solutions

Method	Type
QMIX (Rashid 2018) – monotonic value factorisation	Value decomp.
COMA (Foerster 2018) – counterfactual baseline	Gradient-based
Implicit Credit (Wang 2020) – attention over teammates	Attention
Polarization PG (Chen 2023) – explicit credit via gradient polarity	Explicit

Theme 4 – Partial Observability in Pursuit-Evasion

Why it matters: With full global state, pursuit-evasion becomes a trivial tracking problem. Partial observability forces agents to plan under uncertainty, maintain beliefs, and use memory.

ViPER (Wang, Sartoretti & How, 2025)
Visibility-based PE via RL. Agents observe only within a line-of-sight cone.

Key insight: LOS constraints fundamentally change optimal strategies.

vs. Ours: We add team coordination (3v2) and dynamic topology.

Gonultas & Isler (2024)
PE with dynamic sensor and actuation constraints. Limited range, noise, uncertain speeds.

Key insight: Uncertainty about opponent dynamics is as important as sensing range.

vs. Ours: Our memory-augmented obs addresses uncertainty at a system level.

Our pipeline: vector state + 360° raycasts (14 Sentinel / 16 Runner rays) + last-known-position memory $\rightarrow$ 117 / 129 float observation vectors

Synthesis – How the Literature Shapes Our Design

Theme	Key Insight	Design Decision
Pursuit-Evasion	Rich behaviors emerge in topologically complex environments	Procedural maze + exit zones + dynamic walls
Cooperative MARL	PPO with parameter sharing is a competitive baseline	Shared policy per team, PPO via Unity ML-Agents
CTDE	Centralized critic resolves non-stationarity	MA-POCA supported; PPO used in reported experiments
Credit Assignment	Shared reward too coarse for indirect contributions	Tactical shaping: pincer, corridor, exit denial
Partial Observability	LOS + memory changes strategy fundamentally	Raycasts + last-known-position; no global state

Problem Formulation

Partially Observable Stochastic Game (POSG)

$\langle N, \mathcal{S}, \{A_i\}, \{O_i\}, T, \{R_i\}, \gamma \rangle$ where N=5, γ=0.99

The joint state covers positions, velocities, and wall configuration. The transition function is governed by Unity's physics engine.

Asymmetric, team-versus-team interaction:

- **Sentinels win** when both Runners are captured before timeout
- **Runners win** when one reaches an exit, or by surviving 120s (5,000 steps)

Zero-sum between teams but cooperative within teams – this is the core reason it is a multi-agent learning problem, not just two single-agent problems.

Environment Design

The environment is built in Unity ML-Agents and structured as **four scenes** of increasing difficulty:

Scene	Walls	Purpose
01_Baseline_OpenArena_3v2	None	Basic pursuit mechanics
02_StaticMaze_3v2	Fixed	Navigation and route-cutting
03_DynamicMaze_3v2	Shift every 8–20s	Adaptation to topology changes
04_Eval_UnseenMaze_3v2	Novel layout	Generalization testing

Dynamic wall mechanics: Each wall is a vertical pillar that can rise or sink. The controller picks two random pillars and toggles them, but only after a safety check that prevents shifting walls within 1.75m of any agent and prevents sealing exits.

Topology mutates during play, but never produces unfair states or unreachable goals.

Agent Observations and Actions

Observation Vector

Component	Sent.	Run.
Self state (pos, vel, heading, alive)	10	10
Env context (time, exit, wall)	7	7
Last-known-position memory	6	6
Rays ($N \times 6$ floats)	84	96
Opponent summary (2×5)	10	10
Total	**117**	**129**

Each ray = 6 floats: normalized distance + one-hot (wall, exit, teammate, opponent, other).

Action Space

2 continuous values, each clamped to $[-1, 1]$, interpreted as 2D world-frame velocity in the XZ plane.

Design Decisions

- **No camera/pixels** – vector observations train 10x faster and are fully interpretable
- **No global state** – agents never see the full maze, making this a true POSG
- **Memory** – stores last-known XYZ when target leaves LOS, decays over time
- **More Runner rays** (16 vs 14) – prey needs wider threat detection

Reward Design

Three reward layers:

1. Terminal Layer
Shared team reward of ± 1.0. Every Sentinel gets $+1.0$ if both Runners captured; every Runner gets $+1.0$ if one escapes or timeout.

2. Shaping Layer
Per-agent: chase/evade progress ($\sim$1e-3/step), plus penalties for wall-loops, orbit-stalls, threat approaches. Mitigates lazy-agent failure mode.

3. Tactical Layer
Coalition-scoped: pincer rewards the 2 Sentinels involved; corridor control rewards blockers; exit denial rewards the guarding Sentinel. Cluster penalty discourages bunching.

Pipeline: YAML config $\rightarrow$ RewardConfig $\rightarrow$ Role-specific Policy $\rightarrow$ RewardEngine $\rightarrow$ agent.AddReward() $\rightarrow$ StepLogger (CSV audit)

Training Algorithm & Architecture

PPO with Parameter Sharing

- 2 policies total (not 5): one Sentinel brain, one Runner brain
- All 3 Sentinels share same weights; both Runners share another
- Coordination emerges from differing observations of the same policy
- 3× sample throughput on Sentinel side per environment step

Network Architecture

Shared-backbone actor-critic MLP:

- Input → 256 units (Swish) → 256 units (Swish)
- Actor head → 2 floats (Gaussian mean +

PPO Hyperparameters

Parameter	Value
Learning rate	3×10^{-4} (linear decay)
Batch size	1024
Buffer size	10240
Hidden layers	2×256
Epochs	3
γ	0.99
λ (GAE)	0.95
ϵ (clip)	0.2
β (entropy)	0.005
Time horizon	64
Max steps	500K
Optimizer	Adam

Follows MAPPO recipe (Yu et al., 2022)

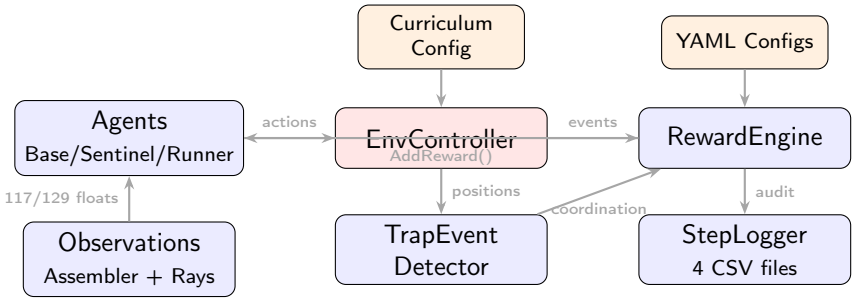
Curriculum

Training is staged. Each stage has both a **minimum-episode threshold** and a **Sentinel-win-rate gate**. Agents advance only when both are satisfied – the curriculum is adaptive, not on a fixed schedule.

Stage	Min Episodes	Walls	Win Rate Gate	Purpose
1. Static, fixed spawns	1,200	None	0.62	Basic pursuit mechanic
2. Static, random spawns	2,200	None	0.55	Prevent route memorizat
3. Dynamic, 20s shifts	3,200	20s	0.50	Slow topology adaptati
4. Dynamic, 8s shifts	3,000	8s	0.45	Fast real-time replannir

Motivation (Zhang et al., 2023): Coordination behaviors are hard to discover from sparse reward in the full-difficulty environment but easier to bootstrap in simpler ones, then transfer.

System Architecture



Key design: Everything is config-driven. Change reward weights, curriculum thresholds, or observation setup without touching C# code.

Agent Hierarchy

Inheritance Chain

- `Unity.MLAgents.Agent`
  - `BaseAgent` – movement, observation, ML-Agents hooks
    - `SentinelAgent` – capture radius, target tracking
    - `RunnerAgent` – escape logic, captured state

Parameter Sharing

- All 3 Sentinels → one PPO brain
- Both Runners → one PPO brain
- Same weights, different observations
- What one agent learns, all agents know

ML-Agents Lifecycle (every step)

1. **Observe** – build the 117/129 float vector from sensors, rays, and memory
2. **Decide** – neural network maps observation → 2 continuous actions
3. **Act** – apply movement to the agent's rigidbody
4. **Reward** – RewardEngine computes and assigns reward
5. **Log** – StepLogger records the event to CSV

Episode boundaries

- Start: agents reset position and memory
- End: both captured, exit reached, or 120s

Logging & Evaluation

4 CSV Output Files

1. `episode_log.csv` – outcome, duration, captured count
2. `agent_step_log.csv` – per-agent XYZ every step
3. `reward_audit.csv` – every reward event with type and value
4. `replay_events.csv` – captures, escapes, tactical events

Evaluation Protocol

- 3 seeds: 42, 101, 202
- Seen layouts (training mazes)
- Unseen layouts (novel procedural mazes)
- Metrics averaged across seeds

Results

Metric	Seen	Unseen
Sentinel WR	62.0%	60.3%
1st capture	17.2s	9.7s
2nd capture	37.8s	24.5s
Escape frac	26.0%	26.8%
Timeout surv	12.0%	12.9%

Key insight:
Win rates transfer (1.7pt drop).
Capture timing shifts (7.5s faster).
Policies are topology-sensitive but not layout-memorized.

Summary

What makes this more than a toy project:

- Modular architecture (agents, rewards, sensors, logging are all separate)
- Config-driven (YAML for everything: rewards, curriculum, PPO, evaluation)
- Full audit trail (4 CSV logs track every reward, every step, every event)
- Reproducible (3 seeds, seen/unseen split, public repo)
- Dynamic environment (walls shift during episodes)

Honest limitations:

- No MA-POCA comparison yet (credit assignment)
- No ablation study (memory on/off, shaping on/off)
- Coordination is observed qualitatively but not quantified with dedicated metrics
- BufferSensor path exists but is untested

Repo:

`github.com/ahmedjawedaj/labyrinth-breach`